This data alone is often misleading. You can tell how many lines of code were changed and by whom, but that doesn't give you an idea of effort or productivity. A junior programmer can make many commits and write a lot of bad code, whereas a senior developer can find a much more concise way to do the same work, so looking at volume doesn't indicate productivity. Additionally, some people like to start hacking away at a problem and hone it as they go; others like to think about it to make sure they get it right the first time. Both methods are perfectly viable ways to work, but the data you'll get from them in this context is totally different.

The data from centralized SCM can be useful when combined with other information, such as how many tasks are getting done from your PTS or whether tasks are moving in the wrong direction. Using those as your indicators of good and bad will help you determine what your SCM data means.

4.3.3. What you can tell from SCM alone

The minimum set of data you can get from SCM is the amount of code everyone on your team is changing. With no other data you can get answers to questions like these:

- ***Who is changing the code?*** —Who is working on what parts of the product? Who is contributing the most change?
- ***How much change is happening in your codebase?*** —What parts of your product are undergoing the most change?

At its core SCM tracks changes to the codebase of your product, which is measured in CLOC. Just as velocity is a relative metric whose number means something different to every team that tracks it, CLOC and LOC are the epitome of relative metrics. LOC varies based on the language your code is written in and the coding style of the developers on the team. A Java program will typically have a much higher LOC than a Rails, Grails, or Python program because it's a much wordier language, but it's even difficult to compare two Java programs without getting into discussions on whose style is better. An example is the following listing.